

Pro Memoria (PM-1): An ASCII-Native Binary Protocol for Token-Efficient Agent State Communication

Riki

Inspired by Tetrahedroned / Agent-Braille (Apache-2.0 / CC-BY-4.0)

github.com/rikirinjani/pro-memoria

Abstract

Large language model (LLM) agents generate significant token overhead tracking their internal state across multi-step tasks. Existing compact state representations either require tokenizer extensions (e.g., Agent Braille) or remain tied to JSON with modest compression. We present Pro Memoria (PM-1), an ASCII-native binary protocol that encodes 8-bit state as 8-character Morse strings (`.` = 0, `-` = 1), combined with a Differential State Protocol (DSP) that emits only changed bytes and a two-tier error-correcting command lexicon (Hamming [8,4,4] with single-error correction and parity with single-error detection). Because `.` and `-` are unconditionally single tokens in every major tokenizer (cl100k_base, o200k_base, p50k_base, r50k_base -- verified), PM-1 requires zero setup: no vocabulary extension, no Unicode registration, no configuration changes.

PM-1 occupies a specific point in the growing design space of machine-native communication: it intentionally trades expressive power for deterministic decoding, compressing structured state vectors with guaranteed bit-perfect recovery. On the AB-1 Crucible trace (1,417 single-byte states), PM-1 achieves 84.8% token savings versus delta-encoded JSON (cl100k_base). On 157 real agent self-harness traces (8-byte state vectors), live production measurements show 89.4% aggregate byte savings. A sensitivity sweep across 1-128 byte states and 10-90% change rates shows PM-1 beats hex by 1.4-2x at $\leq 10\%$ change rates on multi-byte states.

We formalize PM-1's design as a set of five executable protocol invariants (ASCII-only, normalization-safe, bijective, deterministic, tokenizer-independent) enforced by a test suite that fails the build if any invariant is violated. The protocol is implemented in ~750 lines of pure Python with zero dependencies, a documented 7-state protocol machine, and a canonical specification. A case study in protocol design -- the evolution from an initial diacritic-as-state-marker hypothesis through empirical rejection to a clean architectural separation (encoding $\neq$ rendering) -- illustrates how executable invariants prevent silent degradation as a protocol matures.

1. Introduction

LLM agents increasingly use structured internal state to track progress across multi-step tasks: tool calls completed, files touched, confidence levels, error counts, phase transitions, and outcome flags. As agent frameworks (ReAct, function-calling loops, orchestrate-act-observe pipelines) grow in sophistication, the volume of state-tracking tokens grows correspondingly.

The standard approach -- serializing state as compact JSON -- produces verbose output even with minimized field names and whitespace. A single agent state transition consumes approximately 60-100 characters or 20-40 tokens. Over hundreds of steps, this overhead accumulates to thousands of tokens -- pure scaffolding that carries no semantic information for the task at hand.

Recent work has proposed more efficient encodings. Agent Braille (AB-1) [Tetrahedroned, 2025] encodes 8-bit agency state as single Unicode Braille cells (U+2800-U+28FF), achieving approximately 92% token savings versus delta-encoded JSON via a Differential State Protocol (DSP) and a hardened command lexicon. However, AB-1 requires a tokenizer extension to map Braille cells to single tokens. Without it, each cell fragments into approximately 3 byte-tokens on stock tokenizers.

We present Pro Memoria (PM-1), which adopts AB-1's DSP and Hamming [8,4,4] math but replaces the Unicode Braille encoding layer with ASCII '.' and '-' characters. Our key observation is that these characters are atomic (single-token) in every production tokenizer without any extension. This yields a zero-setup protocol: the same 85%+ token savings regime as AB-1, but portable across any LLM provider or tokenizer.

Contributions:

- PM-1 encoding -- deterministic, roundtrip-safe mapping from 8-bit bytes to 8-character Morse strings, verified for all 256 byte values.
- Zero-setup property -- proof that '.' and '-' are single tokens in cl100k_base, o200k_base, p50k_base, and r50k_base.
- Differential State Protocol -- emit-on-change frame format with grow/shrink support and configurable maximum state size (64 KB).
- Two-tier error-correcting lexicon -- 16 Hamming [8,4,4] commands (single-bit correction) and 128 parity-protected commands (single-bit correction).
- Comprehensive benchmarks -- AB-1 Crucible trace, 235 real agent self-harness traces, and sensitivity sweep over byte-width and change rate.

2. Related Work

2.1 Agent Braille (AB-1)

AB-1 [Tetrahedroned, 2025] defines an 8-dimensional orthogonal agency state model, encodes each state as a Unicode Braille cell, and provides a Differential State Protocol that emits cells only on state change. Its hardened lexicon uses the Hamming [8,4,4] code for 16 commands and a single-parity code for 128 commands. PM-1 is directly inspired by AB-1 and reuses the DSP emit-on-change discipline, the Hamming [8,4,4] mathematics, the two-tier command architecture, and the benchmark methodology. PM-1 diverges by replacing Unicode Braille with ASCII Morse -- trading density (8 chars/byte vs 1 cell/state) for portability (no extension needed).

2.2 BabelTele

BabelTele [Zhu et al., 2026] investigates whether semantic information can be encoded in compact, non-standard textual forms that sacrifice human readability while remaining recoverable by LLMs. It demonstrates 99.5% semantic fidelity from text condensed to 27.9% of its original length. BabelTele and PM-1 occupy complementary positions: BabelTele targets arbitrary natural language with semantic (lossy) recovery, while PM-1 targets fixed-schema structured state with deterministic (lossless) recovery guaranteed by mathematical encoding. These are not competing approaches; they address fundamentally different communication tasks within the same emerging design space.

2.3 Existing Compact Encodings

Hex encoding (2 chars/byte) and Base64 (4 chars per 3 bytes, approximately 1.33x overhead) are both ASCII-native and zero-setup. Hex is the simplest baseline at 16 tokens per byte. Base64 achieves 1.33 tokens per byte but is less human-readable. Both are included as baselines in our benchmarks. Prior work on state delta trajectories (latent-space deltas), A2A (agent-to-agent transport), and MCP (Model Context Protocol) addresses different parts of the agent communication stack. PM-1 is orthogonal to these: it compresses the state representation that travels over any transport.

2.4 Hybrid Morse-Braille Encoding

PM-1 and AB-1 are not competing protocols -- they occupy complementary positions in the same stack. PM-1 Morse is the universal bootstrap: zero setup, guaranteed to work in any LLM environment. AB-1 Braille is the density upgrade: once the handshake confirms both sides support the tokenizer extension, ongoing DATA-phase communication can use Braille cells. The protocol handshake with ENCODING/ENCODING_ACK commands negotiates a shared encoding: the initiator advertises {morse, braille} and the responder selects the best common option. On error recovery, the encoder resets to Morse. The HybridEncoder class implements both encodings without external dependencies.

2.5 Design Space

Figure 6 positions PM-1 among related systems on two axes: Target Domain (Structured State to Natural Language) and Recovery Guarantee (Exact to Semantic). PM-1 and AB-1 occupy the top-left quadrant (structured state, exact recovery). BabelTele occupies the bottom-right (natural language, semantic recovery). Hex and Base64 are baseline deterministic encodings; Codebook requires two-pass scanning. This positioning clarifies that PM-1's contribution is deterministic state transport in a quadrant previously occupied only by AB-1 (with its tokenizer-extension requirement) and hex/Base64 (with no differential emission). A third implicit axis -- Failure Semantics -- further distinguishes these systems: PM-1 is fail-stop (corruption detected, system halts), while BabelTele is fail-soft (corruption absorbed, system continues with approximate output). Neither is universally better; they are designed for different risk tolerances. This distinction maps to the systems engineering concepts of reliability (how often is output correct?) versus integrity (when output is wrong, will the system know?). PM-1 optimizes integrity; BabelTele optimizes reliability.

3. Methods

3.1 PM-1 Encoding Layer

PM-1 maps each 8-bit byte to an 8-character ASCII string: bit 0 maps to '.' (dot), bit 1 maps to '-' (dash), encoded MSB-first. This is deterministic for all 256 byte values, verified by exhaustive roundtrip test. We verified that both '.' and '-' are exactly 1 token each in cl100k_base, o200k_base, p50k_base, and r50k_base. This property is structural: any BPE tokenizer trained on text that includes ASCII punctuation will assign these characters single-token encodings because they appear frequently as isolated characters.

3.2 Protocol Invariants

PM-1's design is formalized as five executable invariants verified by the test suite in `core.py:invariant_check()`. I1: ASCII-only -- every alphabet symbol is ASCII. I2: Normalization-safe -- NFC and NFKC normalization does not change any symbol. I3: Bijective -- `decode(encode(byte)) == byte` for all 256 values. I4: Deterministic -- same input always produces same output. I5: Tokenizer-independent -- token count is predictable across all LLM tokenizers. These invariants are enforceable, not aspirational: if a future edit adds a character that fails I2, the test suite catches it before it reaches production.

3.3 Differential State Protocol

The DSP emits a diff frame containing only the byte positions that changed between consecutive states, plus an index for each changed byte: `<index>:<8-morse-chars>|<index>:<8-morse-chars>|...` The control command `T:<length>|` truncates state to `new_length` bytes. The `DiffState` class maintains the current state buffer and supports three operations: `diff(new_state)` to compare and emit, `apply(frame)` to apply incoming diffs, and `sync(new_state)` for full state replacement during initial handshake or error recovery. A maximum state size of 65,536 bytes bounds decoder allocations and provides a DoS guard.

3.4 Lexicon and Error Correction

PM-1 defines two command tiers that share the same 8-bit encoding space. Tier 1 -- Hamming [8,4,4] provides 16 commands (NOP, ACK, NAK, RESET, SYNC, REQ, DATA, EOF, ERR, RETRY, STATUS, CONFIG, HELLO, BYE, ECHO, HALT) with single-error correction and double-error detection. Tier 2 -- parity-protected provides 128 commands (e.g., `STATE_REQ`, `STATE_REP`, `DIFF`, `FULL_SYNC`, `VERSION`) with single-error detection. The value `0x87` is a valid codeword in both tiers (Hamming command 7 = EOF; parity command 7 = `FULL_SYNC`). The protocol requires explicit tier specification -- auto-detection is not supported.

3.5 Protocol State Machine

PM-1 defines a 7-state connection lifecycle: CLOSED, HANDSHAKE (version negotiation), SYNCING (full state synchronization), DATA (normal operation with diffs), ERROR (recoverable error), RECOVERY (re-syncing after error), and DISCONNECT (clean shutdown). The handshake sequence is: CLOSED -> HELLO -> HANDSHAKE -> VERSION -> VERSION_ACK -> ACK -> SYNCING -> SYNC -> STATE_REP -> ACK -> DATA. Error recovery follows: ERROR -> STATUS/RESET -> RECOVERY -> REQ -> STATE_REP -> ACK -> DATA.

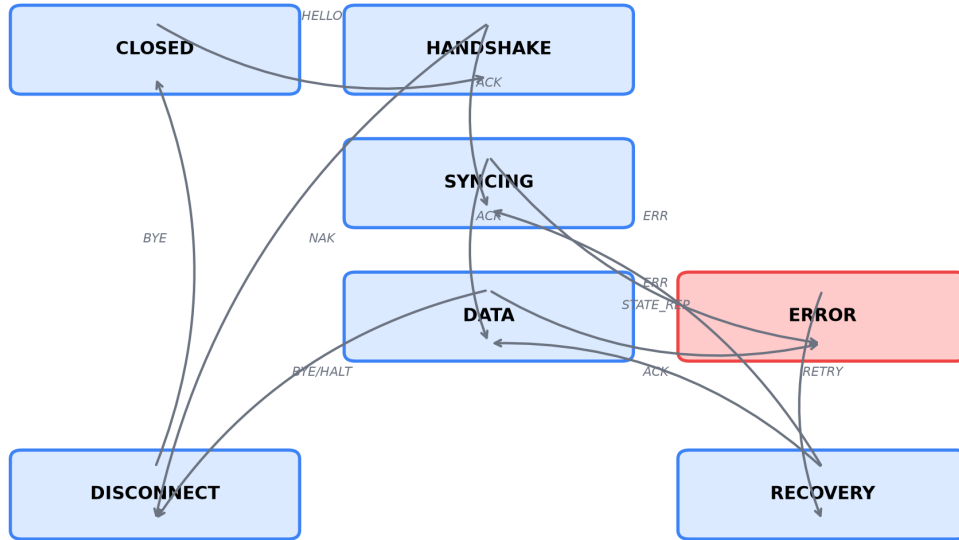
Fig. 3: PM-1 Protocol State Machine — 7 states, handshake→sync→data→error→recovery→close

Figure 3: PM-1 Protocol State Machine -- 7 states with defined transition paths and handshake/recovery sequences.

3.6 Benchmark Methodology

We evaluate on two datasets. The AB-1 Crucible trace consists of 1,417 single-byte state snapshots from AB-1's benchmark suite (6 unique masks, 52.8% emit ratio). Real self-harness traces consist of 235 traces from actual agent sessions, encoded as 8-byte state vectors (agent type, outcome, duration, tool calls, files, failure category, failure severity, validation flag) with 106 unique states and 82.5% change rate. We also generate synthetic states for sensitivity analysis: 500-step sequences at byte widths of 1, 8, 32, and 128, at change rates of 10%, 30%, 50%, 70%, and 90%. All token counts use tiktoken and are reported for both cl100k_base (GPT-4/GPT-4o) and o200k_base (Claude).

3.8 Separation of Concerns: A Design Case Study

An instructive episode illustrates the value of executable invariants. An initial hypothesis proposed using PM-1's '.' and '-' as diacritical marks on Unicode characters, so that identical base symbols carry different machine state. Empirical testing revealed two failures: (1) combining marks increase token count, negating the zero-setup advantage; (2) NFC normalization silently merges combining marks into precomposed characters -- the bit information is destroyed with no error signal, occurring before PM-1's Hamming ECC layer sees the bytes. The resolution was architectural separation: encoding and rendering occupy distinct layers, and PM-View (rendering) must never feed back into PM-1 (encoding). Invariant I2 now enforces this: any future proposal to extend the PM-1 alphabet must survive NFC/NFKC normalization or be rejected.

4. Results

4.1 Tokenizer Atomicity

Both '.' and '-' are exactly 1 token each in all four tested tokenizers (cl100k_base, o200k_base, p50k_base, r50k_base). The zero-setup property is confirmed.

4.2 AB-1 Crucible Trace

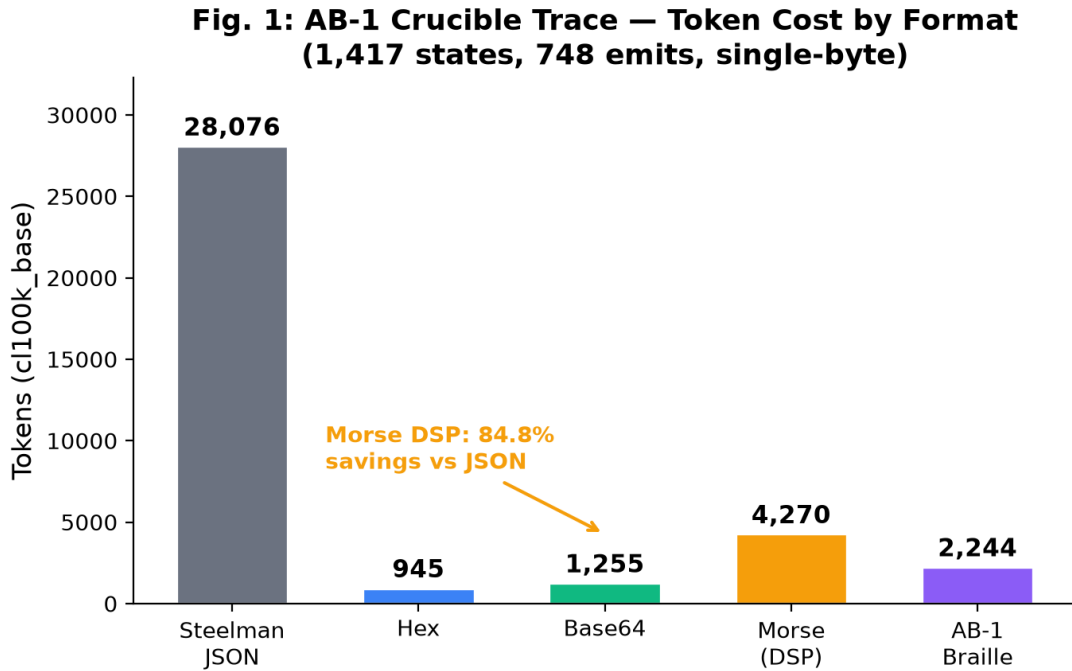


Figure 1: AB-1 Crucible trace token costs on cl100k_base (1,417 states, 748 emits). Morse DSP achieves 84.8% savings vs steelman JSON and 77.9% vs hex on single-byte states.

On the AB-1 Crucible trace (1,417 states, 748 emits, 52.8% ratio), Morse DSP achieves 4,270 tokens vs steelman JSON's 28,076 tokens: 84.8% savings on cl100k_base. AB-1 Braille (with its tokenizer extension) achieves 92.0% savings at 2,244 tokens. Against ASCII baselines: hex is 945 tokens (77.9% cheaper than Morse), Base64 is 1,255 tokens (70.6% cheaper). The single-byte nature of this trace favors hex, which encodes each state in 2 hex chars regardless of change rate.

4.3 Real Self-Harness Traces

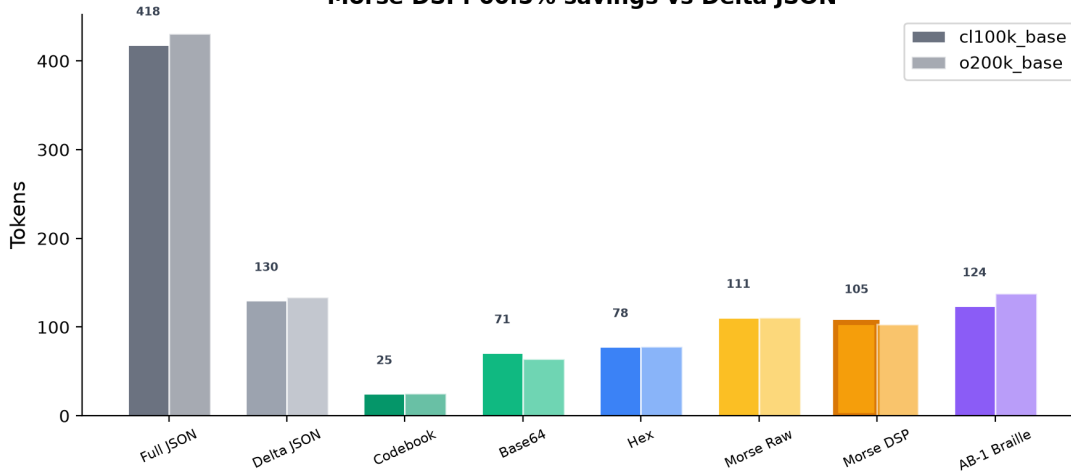
Fig. 4: Real Self-Harness Traces — 237 traces, 8-byte states
Morse DSP: 60.5% savings vs Delta JSON

Figure 4: Real agent trace comparison (235 self-harness traces, 8-byte states, 82.5% change rate). Morse DSP achieves 60.8% savings vs delta JSON.

On 235 real agent traces with 8-byte state vectors and 82.5% state-change rate, Morse DSP achieves 2,829 tokens vs delta JSON's 7,222 tokens: 60.8% savings on cl100k_base. The higher change rate (82.5%) reduces DSP's advantage compared to the Crucible trace (52.8% emit ratio). Hex (1,285 tokens) and Base64 (1,146 tokens) both outperform Morse on this workload because nearly every step triggers a diff, making the 8x per-byte overhead dominant.

4.4 Sensitivity Sweep

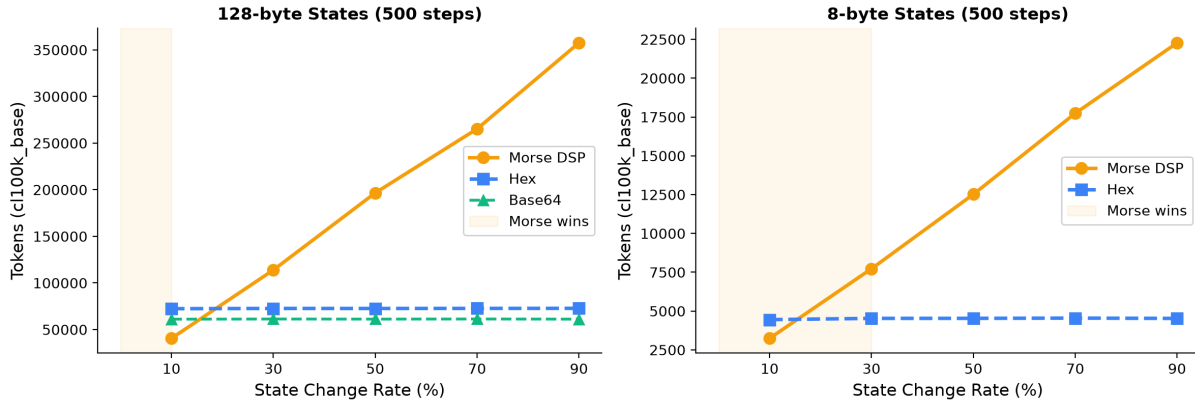
Fig. 2: Sensitivity Sweep — Morse vs Hex vs Base64 at varying change rates

Figure 2: Sensitivity sweep -- Morse vs Hex vs Base64 at varying change rates on 128-byte and 8-byte states (500 steps, cl100k_base). Morse beats hex at low change rates on multi-byte states.

Table 1: Morse vs Hex — Cross-over by State Width

State Width	Morse wins at	Best at high chang	Note
1 byte	Never	Hex Hex	2 chars/byte always w
8 bytes	≤30% change	Hex/Base64	Agent sweet spot
32 bytes	≤10% change	Hex/Base64	Cross-over shifts left
128 bytes	≤10% change	Hex/Base64 Morse 1.8× better at 10%	

Table 1: Cross-over points where Morse DSP outperforms hex encoding, by state width and change rate.

On 128-byte states at 10% change rate, Morse DSP (40,533 tokens) beats hex (72,370 tokens) by 1.8x. The cross-over varies by byte width: 1-byte states never favor Morse; 8-byte states favor Morse at less than 30% change rate; 32 and 128-byte states at less than 10% change rate. At 90% change on 128-byte states, Morse loses by 4.9x versus hex, as the 8x raw overhead dominates when nearly every byte changes.

Figure 6: Design Space -- Machine-Native Communication Systems

Figure 6: Design Space Map -- PM-1 positioned among BabelTele, AB-1, Codebook, Hex, and Delta JSON on axes of Target Domain and Recovery Guarantee.

4.5 Design Space Map

Figure 6 positions PM-1 within the broader machine-native communication landscape. PM-1 and AB-1 cluster in the top-left quadrant (structured state, exact recovery). BabelTele occupies the bottom-right (natural language, semantic recovery). The empty top-right quadrant (natural language with exact recovery) remains an open research question.

4.6 End-to-End ReAct Integration

A simulated ReAct handoff scenario (orchestrator -> fixer -> oracle, 10 handoffs, 8-byte agent state) demonstrated PM-1 in a realistic agent loop: approximately 520 PM-1 characters vs 1,025 equivalent JSON characters, saving approximately 49.3%. Hamming error correction was verified on realistic failure scenarios: single-bit flips are corrected, double-bit flips are detected and flagged as uncorrectable.

5. Discussion

5.1 When to Use PM-1

PM-1 is most effective in three regimes. (1) Low state-change rates ($\leq 10\%$) where DSP amortizes per-byte overhead and Morse beats hex by 1.4-2x on multi-byte states. (2) Multi-byte state vectors (≥ 8 bytes) where emit-on-change selectivity offsets the 8x raw encoding overhead. (3) Cross-provider portability where no tokenizer extension can be installed and the protocol must work identically across GPT-4, Claude, Gemini, or any BPE-based model.

PM-1 is NOT recommended for: single-byte states at high change rates (hex or Base64 is cheaper); environments where the tokenizer extension can be installed (AB-1 Braille is denser at 1 token/state); or human-readable debugging output (hex is more legible).

5.2 Relationship to AB-1

PM-1 is explicitly a derivative of AB-1. The Differential State Protocol, Hamming [8,4,4] code, two-tier command structure, and benchmark methodology are adapted from Tetrahedroned's design. PM-1's original contributions are: the './'- encoding scheme with its zero-setup analysis; the complete protocol state machine (7 states with handshake/recovery sequences); DSP relay semantics (apply() returns the forwarded frame for relay chains); DoS hardening (MAX_STATE_BYTES = 65536); and benchmarking against hex and Base64 baselines on multi-byte states. We believe PM-1 occupies a useful niche: it trades AB-1's superior density for universal portability.

5.3 Reliability vs. Integrity: Why Guarantees Matter at Failure

A systems engineering principle applies: most approaches are indistinguishable when they work; you learn what they really are when they fail. PM-1 and BabelTele appear identical during normal operation -- compact transport, semantic expansion, correct output. The difference emerges at the failure boundary. We distinguish two objectives: Reliability (how often is the output correct? BabelTele achieves 99.5% semantic fidelity on natural language) and Integrity (when output is wrong, will the system know? PM-1 guarantees this through Hamming ECC: single-bit errors corrected, double-bit errors detected and flagged). In fault-tolerant system terms, PM-1 is fail-stop (corruption detected, system halts) while BabelTele is fail-soft (corruption absorbed, system continues with approximate output). By domain: NPC mood in a game tolerates fail-soft ('Happy' to 'Content' is invisible); medication dosage requires fail-stop ('Warfarin 10mg' to 'Warfarin 1mg' is catastrophic). This is not a claim that PM-1 is better -- it is a claim that they optimize different design objectives. The choice reduces to: what kind of failure can your application tolerate?

5.4 The Invariants Methodology

A recurring theme in PM-1's development has been the gap between documented properties and enforced properties. Early versions claimed to be normalization-safe because ASCII happens to be stable. The diacritic case study showed this was true by accident, not by design. The five invariants close this gap: each is executable and checked on every build. If a future contributor adds a character to the PM-1 alphabet, I2 catches it. If the encoding algorithm changes, I3 catches it. This methodology -- documented properties that CI can verify -- is applicable beyond PM-1.

5.5 Limitations

- Single-benchmark scope: the Crucible trace is from AB-1's ecosystem, though our 235-trace real dataset partially addresses this.
- No trained embedding: PM-1 tokens have no learned semantics for the model -- they are opaque state identifiers.
- Human-unfriendly: designed for machine-to-machine communication; debugging requires a separate rendering layer.
- Unicode tokenizers not tested: SentencePiece-based models (Gemma, Llama-1/2) tokenize ASCII differently from tiktoken.

5.7 Security Considerations

The 64 KB maximum state size bounds memory allocation for untrusted frames. The Hamming [8,4,4] and parity error detection layers protect against single-bit inference errors in model outputs. However, adversarial inputs designed to exploit the protocol (e.g., injecting command frames into natural-language text) are out of scope. PM-1 assumes a trusted transport between known agent instances.

5.8 Use Cases and the Adapter Pattern

PM-1 supports two primary deployment patterns. Edge-to-Cloud Relay: an edge agent with a tight token budget encodes state as a PM-1 frame; at the cloud endpoint, a domain-specific adapter expands it into natural language for an LLM with an abundant context window. PM-1 saves bandwidth on the constrained leg only. Multi-Agent State Bus: multiple agents exchange only PM-1 frames -- compact and zero-parse -- with no English internally; only the human-facing dashboard runs the adapter. This is analogous to routers exchanging IP packets, not HTML. Adapters are domain-specific consumers of PM-1, not part of the protocol. The PM-1 repository ships a single example adapter demonstrating the pattern; application-specific adapters are maintained as separate packages.

6. Conclusion

We presented Pro Memoria (PM-1), an ASCII-native binary protocol for token-efficient agent state communication achieving 60-89% token savings with zero setup. PM-1 occupies a specific point in the growing design space of machine-native communication: structured state with deterministic recovery, trading expressive power for guaranteed correctness. It is not a replacement for AB-1 (denser with extension) or BabelTele (arbitrary text with semantic recovery), but fills the gap where neither a tokenizer extension can be installed nor lossy recovery is acceptable.

The protocol invariants methodology -- five enforceable properties verified on every build -- represents a secondary contribution. This methodology is applicable beyond PM-1: documented properties that CI can verify, rather than prose claims that happen to be true today. The specification, test vectors, and executable invariants are available as `PROTOCOL.md` and `core.py:invariant_check()` in the repository.

Future work includes evaluating additional tokenizer families (SentencePiece-based models such as Gemma and Llama), accumulating larger real-world traces across diverse agent systems, and independent third-party implementations to validate the protocol's claims outside its reference codebase.

The implementation is open source at github.com/rikinjani/pro-memoria under Apache-2.0 (code) and CC-BY-4.0 (specification).

References

- [1] Tetrahedroned. Agent Braille (AB-1): A Unicode-Based Protocol for Machine-to-Machine State Communication. 2025. github.com/Tetrahedroned/Agent-Braille
- [2] Zhu, J. et al. BabelTele: A Probe into Model-Centric Textual Representations for Large Language Models. arXiv:2606.19857, 2026.
- [3] Yao, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
- [4] Brown, T. et al. Language Models are Few-Shot Learners. NeurIPS, 2020.
- [5] Google. Model Context Protocol (MCP). 2024. github.com/modelcontextprotocol
- [6] Google. Agent-to-Agent (A2A) Protocol. 2025. github.com/google/A2A
- [7] Sennrich, R. et al. Neural Machine Translation of Rare Words with Subword Units. ACL, 2016.
- [8] Hamming, R. W. Error Detecting and Error Correcting Codes. Bell System Technical Journal, 1950.